

Bluetooth Fundamentals

Bluetooth Fundamentals

1. Bluetooth Overview
 - 1.1 What is Bluetooth
 - 1.2 Bluetooth Version Evolution
 - 1.3 Bluetooth Frequency Bands and Channels
 - 1.4 Bluetooth vs WiFi
 2. Bluetooth Protocol Stack Architecture
 - 2.1 Dual-Mode Architecture
 - 2.2 Host vs Controller Layering
 - 2.3 Layer Details
 3. Classic Bluetooth vs Low Energy Bluetooth
 - 3.1 Core Differences
 - 3.2 Why BLE Power Consumption is Low
 4. BLE Core Concepts
 - 4.1 GAP — Generic Access Profile
 - Four GAP Device Roles
 - Advertising Types
 - Advertising Data Format
 - 4.2 ATT — Attribute Protocol
 - Attribute Structure
 - GATT Operations and Permission Correspondence
 - 4.3 GATT — Generic Attribute Profile
 - UUID Structure
 - CCCD — Client Characteristic Configuration Descriptor
 - 4.4 Notify vs Indicate Comparison
 - 4.5 BLE Connection Parameters
 5. BLE Advertising and Connection Process
 - 5.1 Complete Lifecycle
 - 5.2 Advertising Interval and Discovery Delay
 - 5.3 BLE 5.0 Extended Advertising
 6. SM — Security Manager
 - 6.1 Pairing Methods
 - 6.2 Three Phases of Pairing
 7. Classic Bluetooth Profile Introduction
 8. ESP32-S3 Bluetooth Hardware Features
 - 8.1 Bluetooth and WiFi Coexistence — 2.4GHz Resource Arbitration
 - 8.2 Coexistence Configuration
 9. ESP-IDF Bluetooth Protocol Stack Selection
 - 9.1 Bluedroid vs NimBLE
 - 9.2 Protocol Stack Selection
 - 9.3 Bluedroid menuconfig Key Configuration
 10. Common Troubleshooting
-

1. Bluetooth Overview

1.1 What is Bluetooth

Bluetooth is a short-range wireless communication technology operating in the **2.4 GHz ISM band** (2400~2483.5 MHz), used for **近距离** data exchange between devices. The name comes from 10th century Nordic King Harald Bluetooth (Harald Blåtann),**寓意** unifying communication between different devices.

Key Parameter	Description
Frequency Band	2.4000 ~ 2.4835 GHz (ISM license-free band)
Modulation	GFSK (Gaussian Frequency Shift Keying), BLE 1M / 2M physical layer
Frequency Hopping	Adaptive Frequency Hopping (AFH), 1600 hops/s (classic) / pseudo-random hop (BLE)
Communication Distance	Typical 10m (Class 2, 0~4 dBm), max 100m (Class 1, +20 dBm)
Network Topology	Point-to-point (Piconet) / Broadcast / Mesh (BLE Mesh)

Bluetooth name comes from Danish King Harald "Bluetooth" Gormson, who unified Scandinavian tribes —as Bluetooth technology aims to unify short-range communication protocols between different devices.

1.2 Bluetooth Version Evolution

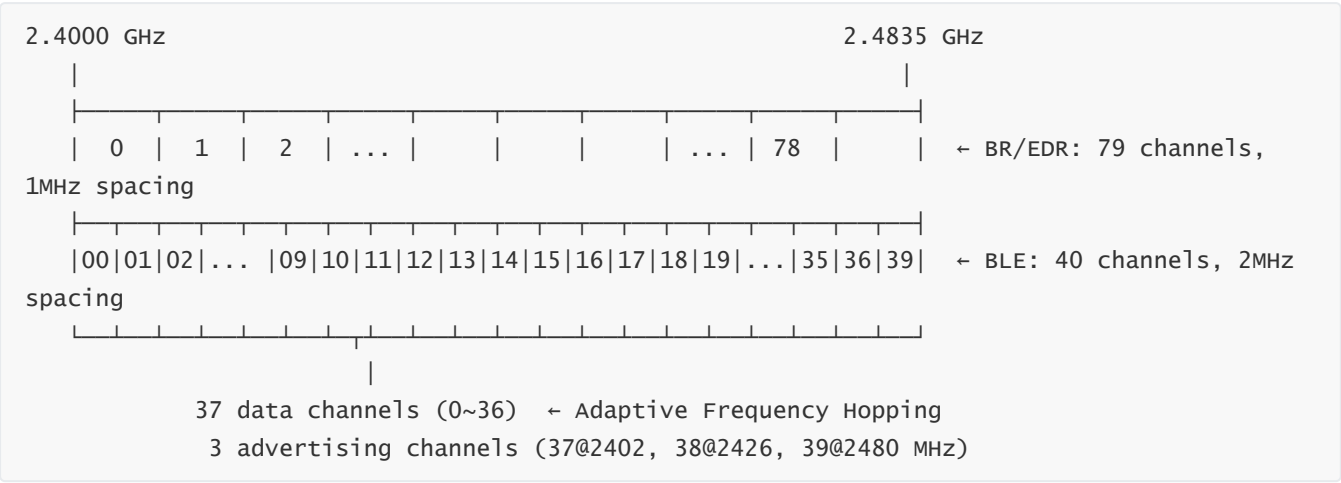
Version	Year	Key Features	Max Speed
1.0 / 1.0B	1999	First commercial version, serious interoperability issues	721 kbps
1.1 / 1.2	2001~2003	AFH adaptive frequency hopping, reduce WiFi interference	721 kbps
2.0 + EDR	2004	Enhanced Data Rate, $\pi/4$ -DQPSK / 8DPSK modulation	2.1 Mbps
2.1 + EDR	2007	Secure Simple Pairing (SSP), reduce pairing complexity	2.1 Mbps
3.0 + HS	2009	Introduced 802.11 (WiFi) as high-speed channel	24 Mbps
4.0	2010	BLE first introduced , low power mode, coin battery can work for months	1 Mbps (BLE)
4.1 / 4.2	2013~2014	BLE enhanced: LE Secure Connections, data length extension	1 Mbps
5.0	2016	2M PHY, LE Long Range (Coded PHY), advertising extensions	2 Mbps
5.1	2019	AoA / AoD direction finding (centimeter-level)	2 Mbps

Version	Year	Key Features	Max Speed
5.2	2020	LE Audio, LC3 codec, Isochronous Channels	2 Mbps
5.3 / 5.4	2021~2023	Channel classification enhancement, periodic advertising enhancement, PAwR	2 Mbps

ESP32-S3 supports **BLE 5.0** and **Classic Bluetooth (BR/EDR)**, theoretical max speed: ~1400 kbps application layer throughput under BLE 2M PHY.

1.3 Bluetooth Frequency Bands and Channels

2.4GHz ISM band divides different channel grids by Bluetooth type:



Bluetooth Type	Channels	Channel Spacing	Hop Rate
BR/EDR (Classic Bluetooth)	79	1 MHz	1600 hop/s
BLE (Low Energy)	40 (37 data + 3 advertising)	2 MHz	Adaptive pseudo-random

BLE intentionally places advertising channels at 2402 / 2426 / 2480 MHz—these three frequencies avoid WiFi's most commonly used channels 1, 6, 11 (2412 / 2437 / 2462 MHz), physically reducing mutual interference with WiFi.

1.4 Bluetooth vs WiFi

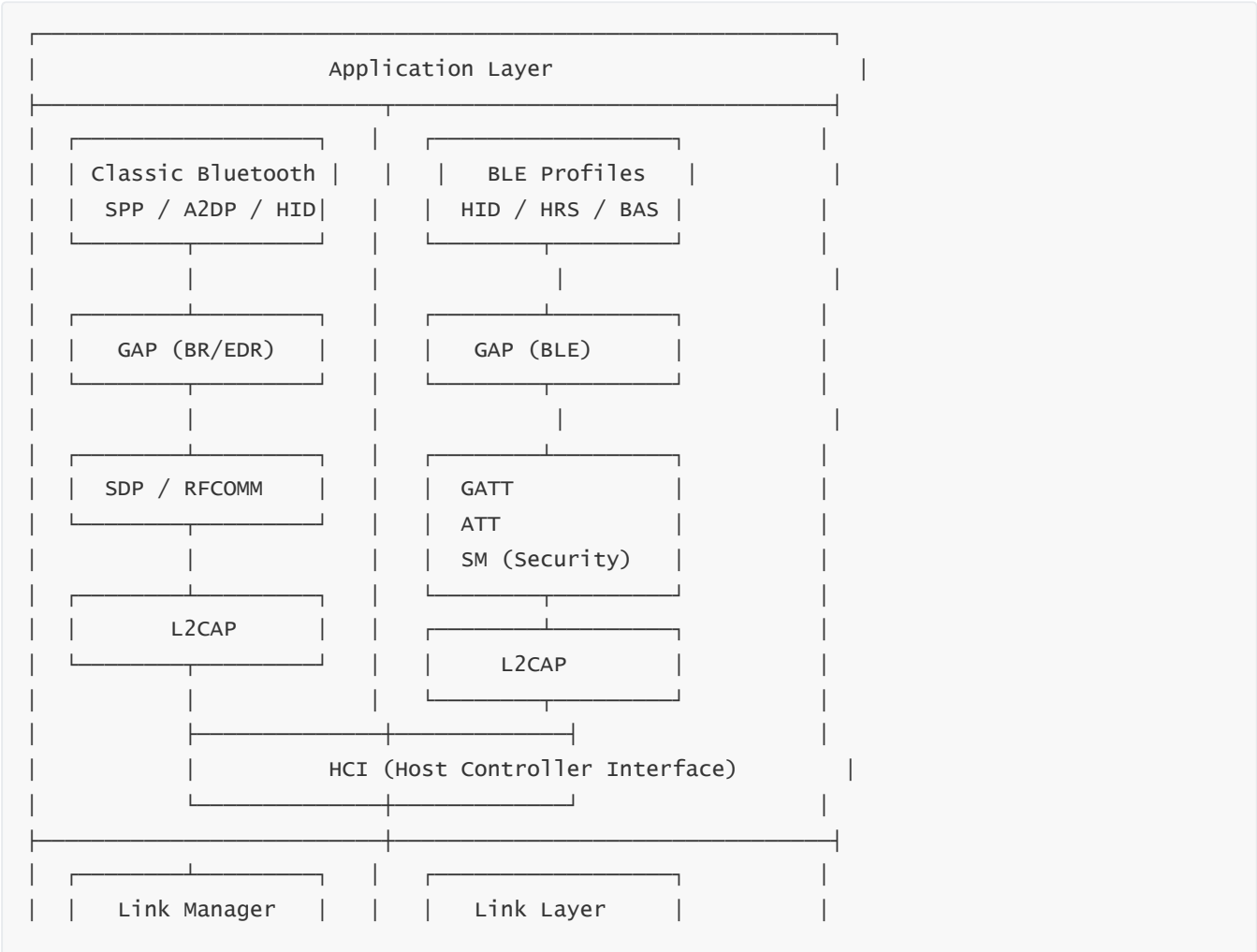
Comparison	Bluetooth (BLE 5.0)	WiFi (802.11n)
Frequency Band	2.4 GHz	2.4 GHz / 5 GHz
Channel Bandwidth	2 MHz	20 / 40 MHz
Max application layer speed	~1.4 Mbps	~50 Mbps (ESP32-S3)

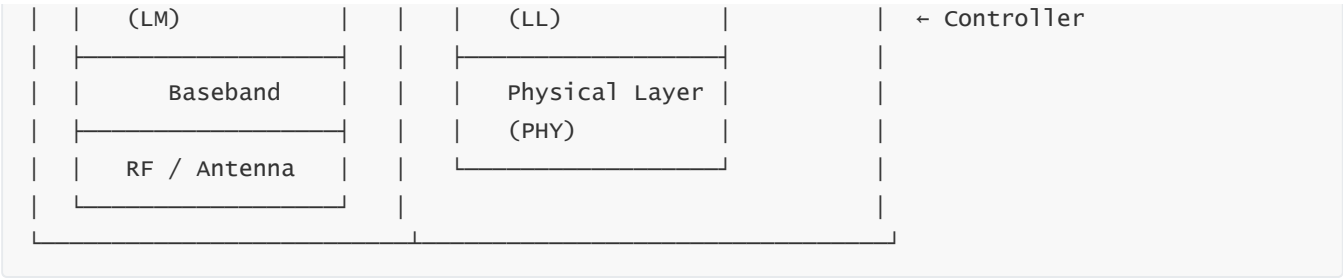
Comparison	Bluetooth (BLE 5.0)	WiFi (802.11n)
Power consumption	Extremely low (μ A level sleep)	Relatively high (mA level continuous receive)
Connection establishment	3 ms	100 ms ~ several seconds
Typical communication distance	10~100m	30~100m
Network topology	Point-to-point / Broadcast / Mesh	Star (AP + STA)
Applicable scenarios	Wearables, sensors, audio	Internet, video streaming, OTA
IP protocol stack	None (needs upper layer like 6LoWPAN)	Native TCP/IP

2. Bluetooth Protocol Stack Architecture

2.1 Dual-Mode Architecture

ESP32-S3 supports **dual-mode Bluetooth**—the same chip can run classic Bluetooth (BR/EDR) and BLE simultaneously. Both share RF front-end but have independent protocol stacks:





2.2 Host vs Controller Layering

Bluetooth protocol stack is divided into two major parts, communicating through **HCI (Host Controller Interface)**:

Layer	Sub-modules	Responsibilities
Host	GAP / GATT / ATT / SM / L2CAP / SDP / RFCOMM	Protocol logic, security management, service discovery
HCI	UART / USB / SDIO transport	Standard communication interface between Host ↔ Controller
Controller	Link Layer / Baseband / PHY / RF	RF control, baseband processing, frequency hopping, physical layer modulation/demodulation

On ESP32-S3, Host and Controller run on the same chip, HCI communicates via internal virtual interface (not physical UART). On phones and other devices, Host typically runs on main CPU, Controller is a separate Bluetooth chip.

2.3 Layer Details

Layer	Full Name	Function	Key Concepts
PHY	Physical Layer	Radio transceiver, GFSK modulation	1M / 2M / Coded PHY
LL	Link Layer	Advertising, scanning, connection establish/maintain, hopping	Advertising state / Scanning state / Connection state
HCI	Host Controller Interface	Host ↔ Controller command/event channel	CMD / Event / ACL Data
L2CAP	Logical Link Control & Adaptation Protocol	Data fragmentation/reassembly, protocol multiplexing	CID (Channel ID), MTU
SM	Security Manager	Pairing, binding, encryption key distribution	Just Works / Passkey / OOB
ATT	Attribute Protocol	Read/write/notify protocol for attribute data	Attribute Handle / UUID / Permissions

Layer	Full Name	Function	Key Concepts
GATT	Generic Attribute Profile	Data organization model	Service → Characteristic → Descriptor
GAP	Generic Access Profile	Device discovery and connection management	Advertising / Scanning / Connection / Role definition

3. Classic Bluetooth vs Low Energy Bluetooth

3.1 Core Differences

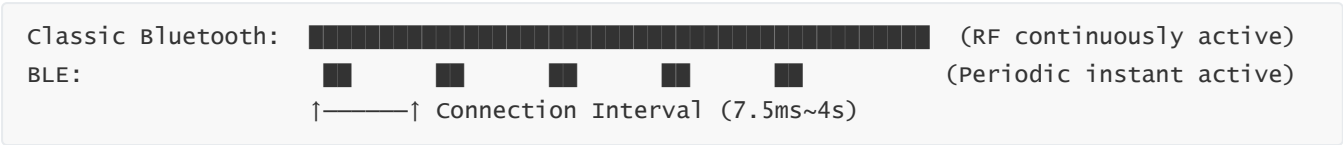
Feature	Classic Bluetooth (BR/EDR)	Low Energy Bluetooth (BLE)
Physical channels	79 × 1MHz	40 × 2MHz
Hop rate	1600 hop/s	Adaptive (can change channel each connection event)
Connection establishment	Scan + Page, ~100ms	Advertising → Scanning → Connection, ~3ms
Max application throughput	2.1 Mbps	~1.4 Mbps (2M PHY)
Power consumption	Medium (~1W active)	Extremely low (~0.01~0.5W, coin battery for months)
Connections	Up to 7 slaves/piconet	No hard limit (protocol stack implementation dependent)
Data model	Serial emulation (SPP/RFCOMM)	Attribute Protocol (ATT/GATT)
Typical applications	Audio streaming, file transfer, Bluetooth headsets	Wearables, sensors, beacons, HID peripherals
Advertising data	Limited (inquiry response)	Rich (up to 31 bytes legacy / 1650 bytes extended advertising BLE 5.0)

3.2 Why BLE Power Consumption is Low

BLE's low power consumption comes from three design choices:

Design Choice	Classic Bluetooth	BLE	Power Impact
Connection duty cycle	Continuous polling	Connection interval min 7.5ms, max 4s	↓ 99% RF active time

Design Choice	Classic Bluetooth	BLE	Power Impact
Packet length	Max 2745 bits	Legacy 376 bits / Extended 2120 bits	↓ Transmit duration
Connection latency	None	Can skip N connection events (Slave Latency)	↓ Slave response count
Advertising	Continuous	Configurable interval 20ms ~ 10.24s	↓ Advertising RF duty cycle



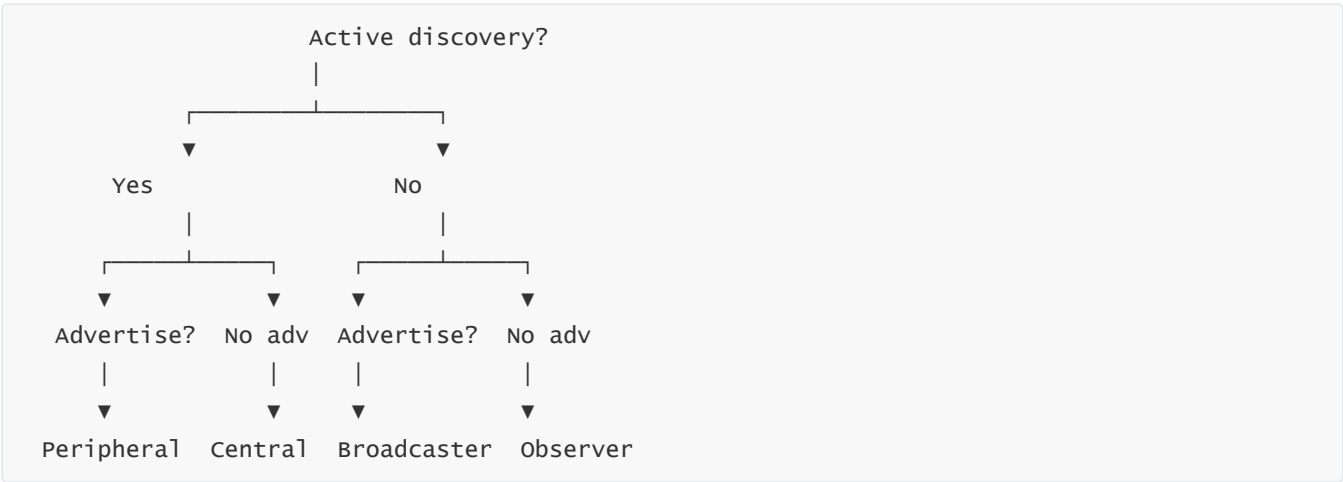
BLE's RF is in **Sleep state** most of the time, only waking for a few milliseconds at predetermined connection events to send/receive data. This is the fundamental reason its power is only 1%~5% of classic Bluetooth.

4. BLE Core Concepts

4.1 GAP — Generic Access Profile

GAP defines how BLE devices **are discovered and connected by other devices**, and is the entry point for all BLE communication.

Four GAP Device Roles



GAP Role	English	Send Advertising	Initiate Connection	Typical Devices
Peripheral	Slave/Peripheral	✓	✗	Sensors, bracelets, beacons

GAP Role	English	Send Advertising	Initiate Connection	Typical Devices
Central	Master/Central	✗	✓	Phones, tablets, gateways
Broadcaster	Broadcaster	✓	✗	iBeacon, Eddystone beacons
Observer	Observer	✗	✗	Passive listeners, packet sniffers

Advertising Types

Advertising PDU Type	Meaning	Connectable	Scannable
ADV_IND	General advertising indication	✓ Connectable	✓ Scannable
ADV_DIRECT_IND	Directed connection advertising (specified target address)	✓ Connectable	✗
ADV_NONCONN_IND	Non-connectable advertising (pure beacon)	✗	✗
ADV_SCAN_IND	Scannable advertising (no connect but can get more data)	✗	✓

Advertising Data Format

Advertising data is organized in **Length-Type-Value (LTV)** structure:

```

Advertising packet (max 31 bytes / legacy mode)
├─ [L=2] [T=Flags]           [V=0x06]           ← 3 bytes
├─ [L=9] [T=Complete Name]   [V="ESP-IDF"]       ← 11 bytes
├─ [L=3] [T=16-bit Service]   [V=0xFFFF0]       ← 5 bytes
└─ [L=...] other fields...
```

AD Type	Value	Description
0x01	Flags	Discoverable mode + BR/EDR support flags
0x02 / 0x03	Incomplete / Complete 16-bit Service UUIDs	Service UUID list
0x08 / 0x09	Shortened / Complete Local Name	Device name
0x0A	TX Power Level	Transmission power (dBm), used for distance estimation

AD Type	Value	Description
0xFF	Manufacturer Specific Data	Manufacturer custom data (used by iBeacon)

4.2 ATT — Attribute Protocol

ATT (Attribute Protocol) is the foundation of BLE data transmission. All data on BLE devices is stored as **attributes (Attribute)**, each attribute is a data unit with handle and UUID.

Attribute Structure

Attribute Handle	UUID	Permissions	Value	
0x0010	0x2800	R	0xFFFF0	← Service declaration
0x0011	0x2803	R	NOTIFY, 0xFFFF1	← Characteristic declaration
0x0012	0xFFF1	R N	"Hello"	← Characteristic value
0x0013	0x2902	R W	0x0001	← CCCD descriptor

Four elements of each attribute:

Element	Description	Example
Attribute Handle	16-bit unique number, allocated by protocol stack	0x0010
Attribute Type (UUID)	Type identifier of attribute	0x2800 = GATT Primary Service declaration
Permissions	Access permissions (read/write/notify/encrypt)	Read + Notify
Value	Attribute value (fixed or variable length)	"Hello World"

GATT Operations and Permission Correspondence

Operation (ATT layer)	GATT Method	Direction	Declaration Required
Read Request / Response	<code>read_cb()</code>	Client ← Server	<code>BLE_GATT_CHR_F_READ</code>
Write Request / Response	<code>write_cb()</code>	Client → Server	<code>BLE_GATT_CHR_F_WRITE</code>
Write Command (no response)	<code>write_cb()</code>	Client → Server	<code>BLE_GATT_CHR_F_WRITE_NO_RSP</code>
Handle Value Notification	<code>notify()</code>	Server → Client	<code>BLE_GATT_CHR_F_NOTIFY</code>

Operation (ATT layer)	GATT Method	Direction	Declaration Required
Handle Value Indication	<code>indicate()</code>	Server → Client (needs confirmation)	<code>BLE_GATT_CHR_F_INDICATE</code>

4.3 GATT — Generic Attribute Profile

GATT (Generic Attribute Profile) is built on ATT, defining the **hierarchical organization model** for BLE data:

```
Profile (Specification/Application layer, e.g. Heart Rate Profile)
|
└─ Service (UUID 16/32/128-bit)
    |   e.g.: 0x180D = Heart Rate Service (SIG standard)
    |       0xFFFF0 = Custom service
    |
    └─ Characteristic (unit that actually exposes data)
        |   UUID + property declaration (Read/Write/Notify/Indicate)
        |   e.g.: 0x2A37 = Heart Rate Measurement (notification-oriented)
        |       0xFFFF1 = Custom notification characteristic
        |
        └─ Descriptor (metadata of characteristic, optional)
            e.g.: 0x2902 = CCCD (Client Characteristic Configuration Descriptor)
                0x2901 = Characteristic User Description (characteristic name readable
text)
```

UUID Structure

UUID Type	Length	Format	Example
16-bit UUID (SIG standard)	2 bytes	<code>0xxxxx</code>	<code>0x180D</code> (Heart Rate Service)
32-bit UUID	4 bytes	<code>0xxxxxxxxx</code>	<code>0x12345678</code>
128-bit UUID (custom)	16 bytes	<code>xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx</code>	<code>4fafc201-1fb5-459e-8fcc-c5c9c331914b</code>

16-bit UUIDs are standardized identifiers allocated by Bluetooth SIG alliance and require purchase authorization. Custom projects can freely use 128-bit UUIDs, or use non-reserved 16-bit UUIDs (like 0xFFFF0~0xFFFE, for development testing only).

CCCD — Client Characteristic Configuration Descriptor

CCCD (Client Characteristic Configuration Descriptor) UUID = `0x2902`, is the most critical descriptor in GATT, controlling slave notification/indication toggle:

Master enables notification: Master → CCCD (0x2902) → write 0x0001 → Slave starts pushing Notify
Master disables notification: Master → CCCD (0x2902) → write 0x0000 → Slave stops pushing

CCCD Value	Meaning
0x0000	Disable all notifications/indications
0x0001	Enable notification (Notify)
0x0002	Enable indication (Indicate)

4.4 Notify vs Indicate Comparison

Comparison	Notify	Indicate
Wait for acknowledgment after sending	✗ No wait	✓ Wait for application layer ACK
Send speed	Fast (no acknowledgment wait)	Slow (wait for ACK each time)
Reliability	Low (lost packets not retransmitted, except link layer retransmission)	High (application layer acknowledgment + retransmission)
Typical scenario	High-frequency sensor reporting (heart rate, temperature)	Critical commands (alarm, switch control)

Notify flow:

Slave →data→ Master
(no ack, continue sending)

Indicate flow:

Slave →data→ Master
←ACK—
Slave sends next only after receiving ACK

4.5 BLE Connection Parameters

After BLE connection is established, master can request modifications to these connection parameters:

Parameter	Range	Default	Description
Connection Interval	7.5ms ~ 4s	~30ms	Interval between two connection events, determines throughput and power consumption
Slave Latency	0 ~ 499	0	Number of connection events slave can skip, reduces slave power consumption
Supervision Timeout	100ms ~ 32s	~2s	How long without receiving data to determine disconnection

Connection interval 7.5ms: ■■■■■■■■ (high throughput, high power)

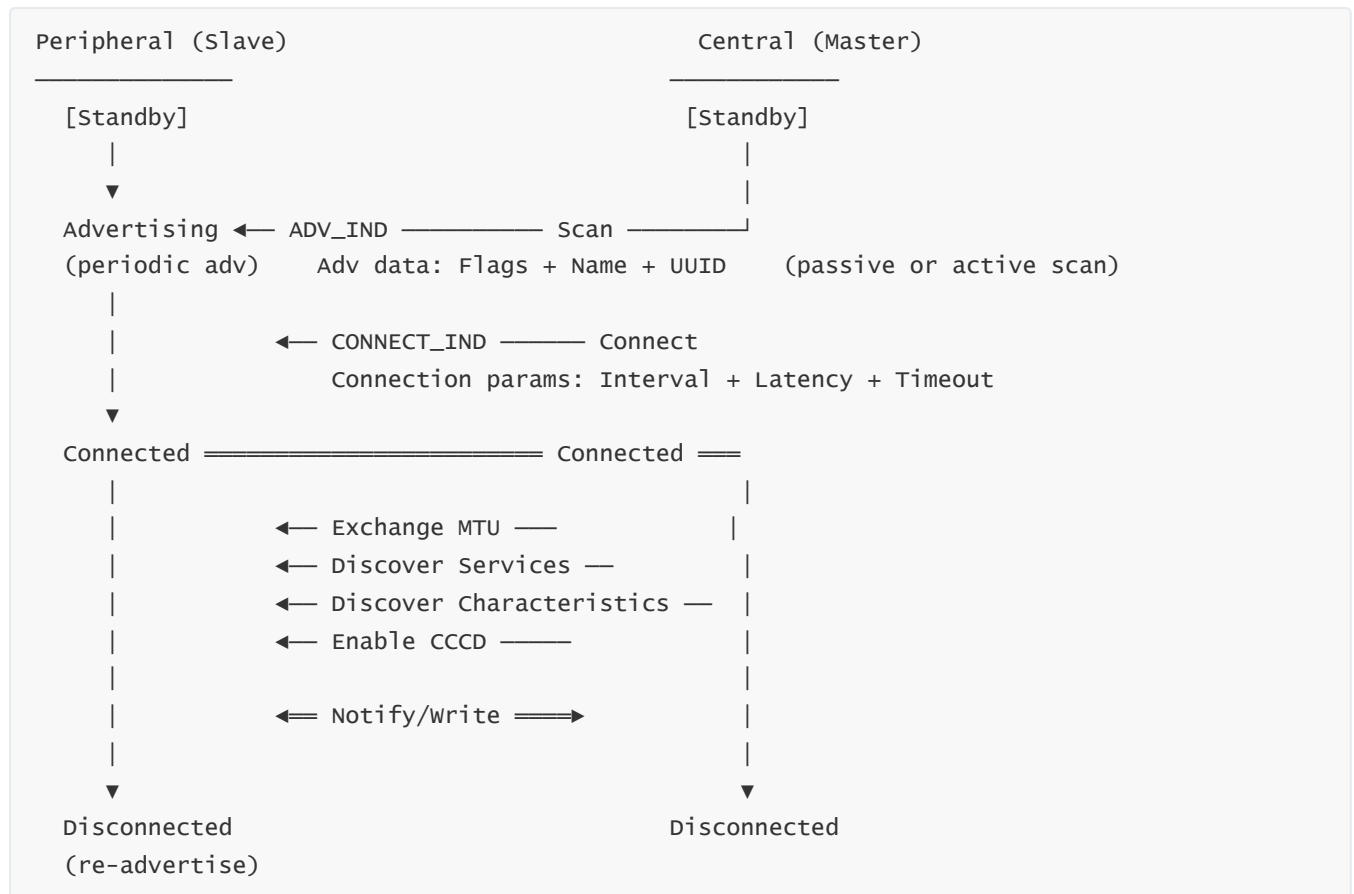
Connection interval 100ms: ■■■■ (medium)

Connection interval 1000ms: ■■ (low throughput, extremely low power)

Connection interval = $7.5\text{ms} \times N$ (N range 6~3200). Smaller value means higher throughput but higher power consumption; larger value means lower power but increased latency. Common sensor scenarios use 100ms~1s; audio scenarios require $\leq 30\text{ms}$.

5. BLE Advertising and Connection Process

5.1 Complete Lifecycle



5.2 Advertising Interval and Discovery Delay

Smaller advertising interval means faster discovery but higher power:

Advertising Interval	Avg Discovery Delay	Avg Power	Applicable Scenario
20ms (fast adv)	~10ms	~1.5mA	Initial provisioning stage
100ms	~50ms	~0.4mA	Normal connectable state
1s (slow adv)	~500ms	~0.05mA	Long standby, beacons

Android/iOS have **scan window limitations**—iOS background scans about once every 1~2s, so advertising interval should not be greater than 500ms, otherwise phone may take a long time to discover the device. Some systems also require specific Service UUID in advertising data for App background filtering.

5.3 BLE 5.0 Extended Advertising

BLE 5.0 introduced extended advertising mechanism, greatly increasing advertising data capacity from 31 bytes:

Legacy advertising (BLE 4.x):

- 3 advertising channels, each ≤ 31 bytes payload

Extended advertising (BLE 5.0):

- Supports carrying advertising data on 37 data channels (not on advertising channels), can chain-send multiple packets (max 1650 bytes payload)

Applications: Indoor navigation, asset tracking, electronic price tag batch updates

6. SM — Security Manager

6.1 Pairing Methods

BLE Security Manager (SM) handles pairing and bonding between devices:

Pairing Method	Security	User Interaction	Applicable Scenario
Just Works	Low (no MITM protection)	No interaction needed	Sensors without UI (temperature/humidity meter)
Passkey Entry	Medium	One party displays 6-digit number, other enters	Keyboard connecting to phone, phone with screen device
Numeric Comparison	High (LE Secure Connections)	Both display 6-digit number, user confirms match	BLE 4.2+ recommended, phone connecting to peripherals
Out of Band (OOB)	Highest	Key passed through external channel like NFC/QR code	Enterprise security, access control systems

6.2 Three Phases of Pairing

Phase 1: Feature Exchange

Both sides exchange IO Capabilities (display/keyboard presence), security requirements
→ Negotiate pairing method (Just Works / Passkey / OOB)

Phase 2: Key Generation

STK (Short Term Key, legacy pairing, BLE 4.0/4.1)
or LTK (Long Term Key, LE Secure Connections, BLE 4.2+)
→ Establish encrypted link

Phase 3: Key Distribution

LTK + IRK (Identity Resolving Key) + CSRK (Connection Signature Resolving Key)
→ Bonding complete, subsequent reconnections auto-encrypt (no pairing again)

Pairing ≠ Bonding. Pairing is a one-time encryption process; bonding stores keys in NVS flash, subsequent connections auto-encrypt—this is the "paired devices list" in Flutter/Android App.

7. Classic Bluetooth Profile Introduction

Although BLE is mainstream for IoT, ESP32-S3 also supports classic Bluetooth, usable for these classic profiles:

Profile	Full Name	Typical Application	ESP32-S3 Support
SPP	Serial Port Profile	Wireless serial passthrough—simulate UART replacing wired serial	✅ Most commonly used
A2DP	Advanced Audio Distribution Profile	Bluetooth audio streaming (headphones, speakers)	✅
AVRCP	Audio/Video Remote Control Profile	Bluetooth remote control (prev/next/volume)	✅
HID	Human Interface Device	Bluetooth keyboard/mouse/gamepad	✅
HFP	Hands-Free Profile	Car hands-free calling	✅

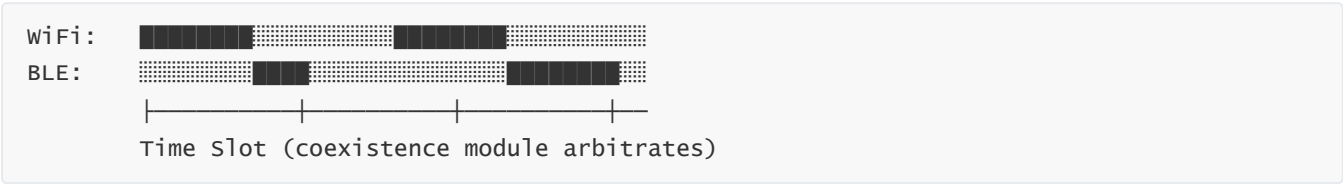
SPP is the most commonly used profile in ESP32-S3 classic Bluetooth development—establishes virtual serial port between two devices via RFCOMM serial emulation protocol, appears to upper layer code as byte stream send/receive, similar to UART but without physical wiring. Applicable to wireless debugging, passthrough control, data acquisition, etc.

8. ESP32-S3 Bluetooth Hardware Features

Feature	Parameter
Bluetooth Standard	BLE 5.0 + BR/EDR (Classic)
Operating Frequency	2.4 GHz ISM
Max TX Power	+20 dBm (Class 1)
Max RX Sensitivity	-97 dBm (BLE 1M)
BLE Max Throughput	~1.4 Mbps (2M PHY, ~700 kbps application layer)
Classic Bluetooth Max Speed	2.1 Mbps (EDR)
Antenna	Onboard PCB ceramic antenna / IPEX external antenna
Protocol Stack	Bluedroid (full) / NimBLE (BLE only)

8.1 Bluetooth and WiFi Coexistence — 2.4GHz Resource Arbitration

Bluetooth and WiFi share the same antenna and 2.4GHz RF front-end—they **cannot transmit simultaneously**. ESP32-S3 resolves conflicts through **Time Division Multiplexing (TDM)**:



Coexistence Strategy	Description
ESP_COEX_PREFER_WIFI	WiFi preferred, Bluetooth actively yields on conflict → BLE throughput decreases but WiFi smooth
ESP_COEX_PREFER_BT	Bluetooth preferred, WiFi TCP ACK may be delayed → BLE connection stable but WiFi may stutter
ESP_COEX_PREFER_BALANCE	Balanced mode, dynamic arbitration

In practice, if BLE connection interval is $\geq 30\text{ms}$ (medium/low throughput scenarios), WiFi and BLE can "peacefully coexist"—BLE occupies predetermined connection event slots (few milliseconds), remaining time WiFi uses RF freely. High-frequency BLE (approaching limit throughput ~700kbps) or dense WiFi transmission (TCP stream, OTA) will interfere with each other; need to tune coexistence strategy or stagger usage periods.

8.2 Coexistence Configuration

Configure coexistence strategy in `menuconfig`:

```
Component config → Bluetooth → Bluedroid → Bluetooth Coexistence
```

Or adjust at runtime in code:

```
// Set coexistence mode to balance
esp_coex_set_preferred_mode(ESP_COEX_PREFER_BALANCE);
```

9. ESP-IDF Bluetooth Protocol Stack Selection

9.1 Bluedroid vs NimBLE

ESP-IDF provides two Bluetooth Host protocol stacks:

Comparison Dimension	Bluedroid	NimBLE
Bluetooth Type	Dual-mode (BR/EDR + BLE)	BLE only
ROM Usage	~512 KB	~32 KB
RAM Usage	~120 KB (minimum)	~4.5 KB
Feature completeness	Full: SPP, A2DP, HID, GATT full features	Lean: GAP + GATT only, BLE 5.0 main features
Configuration complexity	High, many menuconfig options	Low, init ~10 lines of code
Applicable scenarios	Need classic Bluetooth (audio, SPP passthrough) or full BLE features	BLE only, code/ram sensitive scenarios
ESP-IDF default	☒ Default enabled	Need manual selection

9.2 Protocol Stack Selection

```
Need classic Bluetooth (SPP/A2DP/HID)? —Yes→ Bluedroid (only choice)
|
No (BLE only)
|
— Need full BLE features (Mesh, Extended Advertising, LE Audio)?
|   —Yes→ Bluedroid
|
|   — Lean BLE peripheral/central role, code/ram sensitive?
|       —Yes→ NimBLE
```

Subsequent BLE slave and master experiments in this project all use **NimBLE** protocol stack. For IoT teaching scenarios requiring only BLE peripheral/central roles, NimBLE's code and memory overhead is much smaller than Bluedroid, making it a more suitable choice.

9.3 Bluedroid menuconfig Key Configuration

When using Bluedroid, must configure:

```
Component config → Bluetooth
├─ [*] Bluetooth
├─ Bluetooth Host → Bluedroid
├─ Bluedroid Options
│   ├─ [*] Include GATT
│   ├─ [ ] Include Classic Bluetooth (as needed)
│   └─ [*] Enable BLE 5.0 features (as needed)
```

When using NimBLE:

```
Component config → Bluetooth
├─ [*] Bluetooth
├─ Bluetooth Host → NimBLE
├─ Bluetooth Controller → BLE Only
```

10. Common Troubleshooting

Symptom	Cause	Solution
Crash after Bluetooth init	NVS not initialized or partition corrupted	Add try-erase logic to <code>nvs_flash_init()</code> ; or IDE erase entire flash
Advertising start fails <code>rc=12</code>	BLE controller not enabled	menuconfig ensure <code>CONFIG_BT_ENABLED=y</code> , controller mode = BLE Only
Phone can't scan ESP32-S3	Advertising interval too large / no Service UUID	Reduce advertising interval to $\leq 100\text{ms}$; confirm advertising data includes Flags + Name
Disconnects immediately after connection (0~2s)	Connection parameters incompatible / too far	Move closer ($\leq 5\text{m}$); confirm no strong WiFi co-frequency interference
WiFi frequent disconnects during BLE connection	2.4G RF conflict	Adjust coexistence strategy to <code>ESP_COEX_PREFER_WIFI</code> ; or increase BLE connection interval to $\geq 100\text{ms}$
iOS background scan can't find device	iOS doesn't notify App about advertising without Service UUID	Add specified Service UUID in advertising data (iOS App needs to declare corresponding background mode)
Can't discover custom GATT service	NimBLE service table definition error	Confirm <code>ble_gatt_svc_def</code> array ends with <code>{0,}</code> , UUID format correct

Symptom	Cause	Solution
Notify won't send	CCCD not enabled / conn_handle invalid	Confirm master wrote CCCD = 0x0001; confirm conn_handle matches current connection
ATT write not permitted log	Wrote to characteristic value handle without Write permission	Confirm write targets CCCD descriptor handle not characteristic value handle; or add Write flag to characteristic declaration
Bluetooth interrupt during OTA	Underlying restart during OTA causes Bluetooth driver reset	After OTA complete, re- <code>nvs_flash_init()</code> + re- <code>nimble_port_init()</code>